



Algoritmos bio inspirados aplicados ao VRPTW (Vehicle Routing Problem with Time Windows).

Henrique Franco Dalgarrondo e André Lucas Cunha de Almeida

1. O Problema

1.1 Definição

O Problema de Roteamento de Veículos com Janelas de Tempo (Vehicle Routing Problem with Time Windows — VRPTW) é um problema clássico de otimização combinatória com grande relevância prática em logística e distribuição. Ele generaliza o Problema do Caixeiro Viajante (TSP) ao introduzir uma frota de veículos, restrições de capacidade e restrições temporais sobre os atendimentos.

Formalmente, dado um conjunto de n clientes e um depósito central, o objetivo é determinar um conjunto de rotas (cada uma percorrida por um único veículo que parte e retorna ao depósito) de modo que:

- Todos os clientes sejam atendidos exatamente uma vez;
- A demanda total de cada rota não exceda a capacidade do veículo;
- Cada cliente seja atendido dentro de sua janela de tempo;
- O veículo retorne ao depósito dentro da janela de tempo do depósito;
- O número de veículos utilizados seja minimizado (objetivo primário);
- A distância total percorrida seja minimizada (objetivo secundário).

1.2 Dados de Entrada

Campo	Descrição
CUST NO.	Identificador do cliente (0 é o depósito)

Campo	Descrição
XCOORD, YCOORD	Coordenadas geográficas para cálculo de distância euclidiana
DEMAND	Carga necessária para o cliente
READY TIME / DUE DATE	Intervalo de tempo (janela) em que o serviço deve começar
SERVICE TIME	Tempo fixo de atendimento no local

1.4 Função Objetivo

As rotas precisam respeitar a capacidade máxima Q do veículo e as janelas temporais (esperando se o veículo chegar antes do Ready Time, mas sendo proibido chegar após o Due Date). Também é validado, de forma antecipada, se o veículo consegue retornar ao depósito antes do seu fechamento.

Para otimizar os dois objetivos em um único valor (fitness), penalizamos o uso de veículos adicionais usando um peso alto ($\omega = 2000$):

$f(\text{solução}) = \text{distância_total} + 2000 \times \text{num_veículos}$

As distâncias são pré-calculadas em uma matriz $N \times N$ para garantir acesso rápido $O(1)$ e economizar processamento durante a evolução.

Aplicação do algoritmo genético:

2. Modelagem — Giant Tour

2.1 O Problema da Representação

No TSP, um indivíduo é naturalmente representado como uma permutação de cidades — uma sequência que define a ordem de visita. No VRPTW, a solução é composta por múltiplas rotas, o que torna a representação mais complexa: o número de rotas varia entre soluções, e cada rota tem restrições independentes.

Uma abordagem direta seria representar cada rota separadamente, mas isso introduz operadores de cruzamento incompatíveis (pais com número diferente de rotas) e dificulta a herança de material genético entre gerações.

2.2 Giant Tour

A abordagem adotada é o Giant Tour (tour gigante): cada indivíduo da população é representado como uma única permutação de todos os n clientes, sem separadores de rota:

indivíduo = [37, 5, 22, 81, 3, 64, 9, 44, ..., 100]

Esta representação tem comprimento fixo n para todos os indivíduos, independentemente

do número de rotas. O mapeamento entre o giant tour e as rotas reais é realizado por uma função chamada decoder, executada sempre que a solução precisa ser avaliada.

A separação entre representação (giant tour) e solução (rotas) é a essência da abordagem: o AG opera exclusivamente sobre permutações, enquanto o decoder transforma essas permutações em soluções factíveis para o VRPTW.

2.3 Decoder Max-Fill

O decoder implementado é o `decoder_max_fill`, projetado para minimizar o número de veículos antes de considerar a distância. Seu funcionamento é:

1. Inicializa a lista `nao_alocados` com todos os clientes na ordem do giant tour.
2. Abre uma nova rota com o veículo saindo do depósito.
3. Percorre todos os clientes não alocados em sequência e tenta inserir cada um na rota atual — verificando capacidade, janela de tempo e viabilidade de retorno ao depósito.
4. Clientes que não couberem são guardados em restantes.
5. Ao final do percurso, a rota atual é fechada. Os clientes em restantes se tornam os `nao_alocados` da próxima iteração.
6. Repete a partir do passo 2 até que todos os clientes estejam alocados.

A diferença fundamental em relação a um decoder sequencial (que quebra a rota ao primeiro cliente que não cabe) é que o `decoder_max_fill` não descarta os clientes que não cabem imediatamente, ele continua tentando inserir os próximos. Isso permite que um cliente com janela de tempo incompatível com o momento atual seja ignorado, enquanto clientes subsequentes ainda compatíveis entram na rota:

Rota atual: depot → 37 → 5 → 22 (tempo atual = T)

Cliente 81: janela violada → vai para restantes

Cliente 3: testado a partir de 22, tempo T → cabe → entra na rota

Cliente 64: cabe → entra

...

O veículo não se desloca até os clientes rejeitados, o tempo e a posição continuam atualizados apenas para os clientes aceitos.

3. Geração da População Inicial

3.1 Estratégia Mista

Para equilibrar qualidade e diversidade, a população inicial combina:

20% Nearest Neighbor (Vizinho Mais Próximo): Heurística gulosa que agrupa clientes próximos geograficamente, gerando rotas iniciais melhores. Aplicamos pequenas mutações (troca de dois pontos) para criar variações dessa base.

80% Aleatórios: Garante que o algoritmo explore amplamente o espaço de busca, evitando convergência precoce

Os demais indivíduos são gerados por permutação aleatória da lista de clientes:

```
tour = list(range(1, len(customers)))  
random.shuffle(tour)
```

3.4 Justificativa da Estratégia Mista

Uma população 100% aleatória obriga o AG a gastar as primeiras gerações simplesmente descartando soluções muito ruins, antes de começar a explorar regiões promissoras. Uma população 100% baseada em nearest neighbor converge muito rápido mas com pouca diversidade — o AG pode ficar preso em ótimos locais desde o início.

A combinação 20% / 80% fornece um ponto de partida razoável (os indivíduos de nearest neighbor têm fitness consideravelmente melhor que os aleatórios) sem comprometer a diversidade necessária para a exploração ao longo das gerações.

4. Operadores de Cruzamento

O cruzamento é o principal mecanismo de exploração do AG — ele combina material genético de dois indivíduos (pais) para gerar dois novos indivíduos (filhos). Como os indivíduos são permutações, operadores de cruzamento convencionais (como o de ponto de corte de problemas binários) não podem ser aplicados diretamente: trocar segmentos entre dois pais geraria filhos com clientes repetidos ou ausentes.

Três operadores foram implementados, cada um com uma filosofia diferente sobre o que deve ser preservado do pai.

4.1 OX — Order Crossover

O OX (cruzamento por ordem) preserva a ordem relativa dos clientes de um pai. A ideia é que a ordem em que os clientes aparecem no giant tour influencia diretamente quais deles o decoder agrupa na mesma rota — e essa ordenação deve ser herdada.

Sorteiam-se dois pontos de corte aleatórios no cromossomo. O segmento entre os cortes é copiado diretamente do Pai 1 para o Filho 1. As posições restantes são preenchidas com os clientes do Pai 2, na ordem em que aparecem no Pai 2, pulando os que já estão no segmento copiado. O Filho 2 é gerado invertendo os papéis dos pais.

O que o OX preserva: a ordem relativa dos clientes fora do segmento copiado. Se no Pai 2 o cliente A aparece antes do cliente B, essa precedência é mantida no Filho 1.

Limitação para o VRPTW: o OX não tem consciência de quais clientes formam rotas viáveis juntos. Ele pode separar dois clientes que o decoder havia descoberto serem compatíveis, forçando-o a redescobrir essa compatibilidade nas gerações seguintes.

4.2 CX — Cycle Crossover

O CX (cruzamento por ciclo) preserva a posição absoluta dos clientes: cada gene do filho ocupa a mesma posição que ocupava em um de seus pais.

O algoritmo identifica ciclos no mapeamento posicional entre os dois pais. A partir da posição 0, segue-se encadeando posições até que o ciclo se feche. As posições pertencentes ao ciclo recebem os valores do Pai 1; as demais recebem os valores do Pai 2. O Filho 2 é gerado com os papéis invertidos.

O que o CX preserva: a posição absoluta de cada cliente. Um cliente que estava na posição k do Pai 1 continuará na posição k do Filho 1, caso pertença ao ciclo identificado.

Limitação para o VRPTW: preservar posição absoluta não tem significado semântico direto para o decoder — a posição de um cliente no giant tour importa apenas em relação aos seus vizinhos imediatos, não em termos absolutos. O CX tende a gerar menos diversidade que o OX em problemas de permutação com decoder.

4.3 BCRC — Best Cost Route Crossover

O BCRC é o operador mais adequado para o VRPTW dentre os três implementados. Diferentemente do OX e do CX, que operam diretamente no giant tour sem conhecimento das rotas, o BCRC decodifica os pais antes de realizar o cruzamento e herda rotas inteiras de cada pai.

A motivação é direta: se o AG já evoluiu um par de rotas viáveis e de boa qualidade em um pai, faz sentido transmitir esse agrupamento intacto para o filho — em vez de potencialmente desmembrá-lo com um corte de segmento.

Uma rota é escolhida aleatoriamente do Pai 1. Os clientes dessa rota são removidos das rotas do Pai 2. As rotas restantes do Pai 2 são concatenadas em sequência, preservando a ordem interna de cada uma. O giant tour do filho é formado pela rota herdada do Pai 1 seguida do restante do Pai 2. O Filho 2 é gerado com os papéis invertidos.

O que o BCRC preserva: agrupamentos de clientes descobertos pelo AG como compatíveis por capacidade, janelas de tempo e retorno ao depósito. A rota herdada já é uma sequência validada pelo decoder — o filho começa com uma rota garantidamente viável.

5. Dinâmica Evolutiva: Seleção, Mutação e Elitismo

- **Seleção por Torneio:** Escolhe dois indivíduos ao acaso. O melhor vence com 90% de chance e o pior com 10%. Isso mantém a pressão seletiva, mas dá chance a soluções diversas. É muito mais estável numericamente do que a roleta para funções com alta penalidade.
- **Mutação por Inversão (2-opt):** Com uma taxa de 5%, inverte um segmento inteiro do

tour. Isso reorganiza blocos de clientes, permitindo que o decoder crie novos agrupamentos de rotas eficientes.

- **Elitismo:** Garante que os 2 melhores indivíduos de cada geração passem direto para a próxima, evitando a perda acidental de ótimas soluções.

O fluxo completo de uma execução do AG para o VRPTW é o seguinte:

1. Ler instância e calcular matriz de distâncias
2. Gerar população inicial (20% nearest neighbor + 80% aleatório)
3. Para cada geração:
 - a. Decodificar cada indivíduo com `decoder_max_fill`
 - b. Calcular $\text{fitness} = \text{distância} + 2000 \times \text{num_veículos}$
 - c. Registrar melhor da geração
 - d. Selecionar pais por torneio
 - e. Gerar filhos por cruzamento (OX / CX / BCRC)
 - f. Aplicar mutação por inversão
 - g. Avaliar filhos
 - h. Aplicar elitismo para formar nova população
4. Retornar melhor solução encontrada
5. Salvar resultado em arquivo texto e gráfico de convergência

A cada geração, o decoder é chamado para cada indivíduo da população e para cada filho gerado. Não há indivíduos inválidos na população: o decoder garante por construção que todas as restrições de capacidade, janelas de tempo e retorno ao depósito são satisfeitas.

9. Parâmetros do Algoritmo

Parâmetro	Descrição	Valor Padrão
<code>tam_pop</code>	Tamanho da população	100
<code>n_geracoes</code>	Número total de gerações	200
<code>taxa_mutacao</code>	Probabilidade de mutação por indivíduo	0.05
<code>n_elite</code>	Indivíduos preservados por elitismo	2
<code>cruzamento</code>	Operador de cruzamento padrão	OX
<code>peso_veiculo</code>	Penalidade aplicada por veículo na função de fitness	2000.0
<code>frac_nn</code>	Proporção da população inicial via Nearest Neighbor	0.2

10. Teste de parâmetros

Foi feito um teste fatorial variando taxa de mutação, algoritmo de cruzamento, decoder, e tamanho da população. O que gerou 24 configurações, cada uma foi testada com 4 sementes diferentes.

Instância rc208:

=====		
CONFIG	VEIC_MED	DIST_MED

OX_std_mut05_pop100	7.00	1586.4628
OX_std_mut05_pop200	7.75	1236.4851
OX_std_mut10_pop100	7.00	1443.1055
OX_std_mut10_pop200	7.00	1473.2317
OX_maxfill_mut05_pop100	4.00	1289.8664
OX_maxfill_mut05_pop200	4.00	1294.0086
OX_maxfill_mut10_pop100	4.00	1290.4595
OX_maxfill_mut10_pop200	4.00	1273.8645
CX_std_mut05_pop100	7.25	1187.4972
CX_std_mut05_pop200	6.75	1071.3068
CX_std_mut10_pop100	7.00	1224.2678
CX_std_mut10_pop200	7.00	1146.7156
CX_maxfill_mut05_pop100	4.00	1250.5382
CX_maxfill_mut05_pop200	4.00	1215.7589
CX_maxfill_mut10_pop100	4.00	1223.5313
CX_maxfill_mut10_pop200	4.00	1215.5531
BCRC_std_mut05_pop100	4.00	1482.6187
BCRC_std_mut05_pop200	4.00	1434.3821
BCRC_std_mut10_pop100	4.00	1457.7617
BCRC_std_mut10_pop200	4.00	1441.2717
BCRC_maxfill_mut05_pop100	3.75	1337.5810
BCRC_maxfill_mut05_pop200	3.75	1337.5951
BCRC_maxfill_mut10_pop100	3.75	1338.2736
BCRC_maxfill_mut10_pop200	3.75	1336.5684

As configurações que se saíram melhores foram:

BCRC_maxfill_mut10_pop200 (decoder maxfil, cruzamento BCRC, taxa de mutação 0.1, e 200 indivíduos)

OX_maxfill_mut10_pop200 (decoder maxfill, cruzamento OX, taxa de mutação 0.1 e 200 indivíduos)

Nota-se que configurações que tiveram uso de mais veículos conseguiram valores menores de distância total. Porém, como prevalece o critério de utilizar menos veículos, essas soluções não entram como melhores.

Colônia de Formigas Aplicada ao VRPTW

Por que o ACO se encaixa bem neste problema

O Algoritmo de Colônia de Formigas (ACO) possui uma característica natural que o torna particularmente adequado ao VRPTW: ao contrário do AG, que trabalha com uma representação indireta da solução (o giant tour), cada formiga constrói uma solução completa diretamente, tomando decisões cliente a cliente com base nas restrições do problema. Isso elimina a necessidade de um decoder separado e garante que toda solução gerada seja válida por construção.

Além disso, o mecanismo de feromônio permite que a colônia acumule memória coletiva sobre quais sequências de atendimento funcionaram bem nas iterações anteriores, guiando progressivamente as formigas para regiões promissoras do espaço de busca.

Modelagem

A matriz de feromônios $\tau[i][j]$ representa a atratividade de visitar o cliente j imediatamente após o cliente i na mesma rota. O depósito (índice 0) participa da matriz com papel especial: $\tau[0][j]$ indica a atratividade de iniciar uma rota com o cliente j , e $\tau[i][0]$ indica a atratividade de encerrar a rota no cliente i .

A probabilidade de uma formiga escolher o próximo cliente é dada pela combinação de feromônio e heurística de distância:

$$P(j) = (\tau[i][j]^\alpha \cdot \eta[i][j]^\beta) / (\sum_{k \in \text{viáveis}} \tau[i][k]^\alpha \cdot \eta[i][k]^\beta)$$

onde $\eta[i][j] = 1/\text{dist}(i,j)$ favorece clientes mais próximos.

Principais Adaptações ao VRPTW

A adaptação central está na construção da solução: cada formiga não constrói uma única rota, mas um conjunto completo de rotas. A formiga abre uma nova rota quando não há mais clientes viáveis para a rota atual, considerando simultaneamente três restrições antes de inserir cada cliente:

- Capacidade do veículo não excedida
- Chegada dentro da janela de tempo do cliente
- Retorno ao depósito antes do fechamento de sua janela

Ao fim de cada iteração, o feromônio é atualizado com evaporação global seguida de depósito proporcional à qualidade de cada solução: soluções com menos veículos e menor distância depositam mais feromônio, reforçando os arcos que as compõem.

Parâmetros

Parâmetro	Descrição
α	Peso do feromônio na decisão
β	Peso da heurística de distância
ρ	Taxa de evaporação do feromônio
Q	Constante de escala do depósito de feromônio
n_{formigas}	Número de formigas por iteração
τ_0	Valor inicial do feromônio em todos os arcos
peso_veiculo	Penalidade por veículo extra na função de custo

O parâmetro Q deve ser da mesma ordem de grandeza do custo típico das soluções da instância, para que o depósito de feromônio seja proporcional à variação de qualidade entre soluções. Para a instância rc208, custos em torno de 9.000 indicam Q na faixa de 1.000 a 10.000; para instâncias maiores como a c1_2_1, com custos em torno de 120.000, valores de Q entre 10.000 e 100.000 são mais adequados.

Algoritmo Imunológico Clonal (CLONALG)

Aplicado ao VRPTW

Limitações da Abordagem

O CLONALG foi originalmente proposto para problemas de otimização em espaços contínuos, onde pequenas perturbações em um anticorpo produzem pequenas variações no fitness. Essa suposição não se sustenta no VRPTW com representação por giant tour: uma única inversão de segmento pode reorganizar completamente quais clientes o decoder agrupa em cada rota, causando saltos abruptos no fitness. O espaço de busca é altamente

não-linear em relação à representação adotada.

Além disso, o mecanismo de hipermutação proporcional à afinidade perde parte de seu sentido quando o operador de mutação (inversão de segmento) não tem relação direta com a estrutura do problema. No sistema imunológico biológico, mutações acontecem nos aminoácidos do anticorpo e afetam diretamente o sítio de reconhecimento do antígeno. No VRPTW, não existe essa correspondência estrutural entre genes e restrições do problema.

Modelagem Utilizada

Apesar das limitações, o algoritmo foi adaptado mantendo a representação por giant tour, aproveitando a infraestrutura já desenvolvida para o AG. Cada anticorpo é uma permutação de todos os clientes, decodificada pelo `decoder_max_fill` e avaliada pela função de fitness penalizada com $\omega \times \text{num_veículos}$.

O fluxo segue o CLONALG padrão: seleção dos melhores anticorpos, clonagem proporcional ao ranking, hipermutação por inversão de segmento com taxa inversamente proporcional à afinidade, re-seleção dos melhores entre população atual e clones, e substituição dos piores por anticorpos aleatórios para manter diversidade.

Por que o AG é mais adequado neste contexto

O AG com giant tour e `decoder_max_fill` se beneficia diretamente dos operadores de cruzamento (OX, CX, BCRC), que combinam material genético de duas soluções distintas. No CLONALG não há cruzamento — toda a evolução depende exclusivamente de mutação, o que limita a capacidade de combinar agrupamentos de clientes descobertos in indivíduos diferentes.

Parâmetros

Parâmetro	Descrição
tam_pop	Tamanho da população de anticorpos
n_sel	Número de anticorpos selecionados para clonagem
beta	Fator de escala do número de clones por anticorpo
p	Controla a intensidade da hipermutação

Parâmetro	Descrição
d	Anticorpos substituídos por aleatórios a cada geração
peso_veiculo	Penalidade por veículo extra na função de fitness

11. Aplicação do PSO ao VRPTW

11.1 Algoritmo Utilizado

Neste trabalho também foi implementada uma abordagem baseada em Particle Swarm Optimization (PSO) para resolver o Problema de Roteamento de Veículos com Janelas de Tempo (VRPTW).

O PSO é um algoritmo bioinspirado baseado no comportamento coletivo de enxames. Cada partícula representa uma solução candidata para o problema e evolui utilizando informações provenientes da melhor solução encontrada pela própria partícula (pbest) e da melhor solução encontrada pelo enxame (gbest).

Como o VRPTW é um problema combinatório, foi utilizada uma adaptação baseada em Random Keys, permitindo que o algoritmo opere em um espaço contínuo e posteriormente converta as partículas em soluções discretas.

11.2 Representação da Solução

Cada partícula é representada por um vetor de números reais no intervalo $[0,1]$.

Exemplo:

[0.34, 0.91, 0.12, 0.56, 0.48]

Após a ordenação:

Cliente 3 → Cliente 1 → Cliente 5 → Cliente 4 → Cliente 2

A sequência obtida é transformada em uma permutação de clientes que será utilizada pelo decoder para construir as rotas.

11.3 Decoder e Construção das Rotas

Foi utilizado um procedimento de inserção incremental.

Para cada cliente da permutação:

1. Testa-se a inserção em todas as posições das rotas já existentes;
2. Verifica-se capacidade e janela de tempo;
3. Escolhe-se a inserção que produz o menor aumento de distância;

4. Caso nenhuma inserção seja viável, cria-se uma nova rota.

Esse procedimento garante a geração de soluções factíveis durante a avaliação das partículas.

11.4 Operadores Implementados

Atualização da Velocidade

Foi utilizada a equação clássica do PSO:

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(pbest_{ij} - x_{ij}) + c_2r_2(gbest_j - x_{ij})$$

Atualização da Posição

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

Mutação

Quando o algoritmo permanece muitas gerações sem melhoria, são aplicados:

- troca de posições aleatórias;
- reinicialização de genes aleatórios.

Essa estratégia aumenta a diversidade da população.

Busca Local 2-opt

Ao final da execução, a melhor solução encontrada é refinada utilizando o operador 2-opt para reduzir a distância percorrida sem violar as restrições do problema.

10.5 Parâmetros Utilizados

Parâmetro	Valor Padrão
Número de partículas	50
Número máximo de gerações	300
Peso de inércia inicial	0,9
Peso de inércia final	0,4
Coeficiente cognitivo	1,6
Coeficiente social	1,9
Velocidade máxima	0,20
Limite sem melhoria	60
Semente	42

11.6 Estratégia de Penalização

A função objetivo utilizada foi:

$$Fitness = N_{veículos} \times 10^6 + Distância + Penalizações$$

onde:

- soluções com menos veículos têm prioridade absoluta;
- violações de capacidade recebem penalização elevada;
- violações de janelas de tempo recebem penalização elevada;
- excesso de veículos recebe penalização elevada.

Dessa forma, o algoritmo é direcionado para regiões viáveis do espaço de busca.

11.7 Análise dos Resultados

O PSO adaptado foi capaz de gerar soluções viáveis para as instâncias avaliadas.

A representação por Random Keys permitiu utilizar diretamente os operadores clássicos do PSO sem necessidade de operadores específicos para permutações.

A utilização do decoder baseado em inserção incremental garantiu o atendimento das restrições de capacidade e janelas de tempo.

A mutação contribuiu para evitar convergência prematura, enquanto o operador 2-opt reduziu a distância final percorrida.

Os gráficos de convergência demonstraram rápida melhoria nas primeiras gerações, seguida por estabilização gradual da busca.

12. Aplicação da Evolução Diferencial ao VRPTW

12.1 Algoritmo Utilizado

Foi implementada uma abordagem baseada em Evolução Diferencial (Differential Evolution – DE ou ED) para resolver o Problema de Roteamento de Veículos com Janelas de Tempo (VRPTW).

A Evolução Diferencial é um algoritmo evolutivo originalmente desenvolvido para problemas contínuos. Seu funcionamento baseia-se na geração de novos indivíduos por meio da combinação vetorial de soluções já existentes na população.

Como o VRPTW é um problema combinatório, foi utilizada uma adaptação baseada em Random Keys. Cada indivíduo é representado por um vetor de números reais no intervalo $[0,1]$. A ordenação desses valores determina uma permutação dos clientes, que posteriormente é convertida em um conjunto de rotas por um procedimento de decodificação.

Diferentemente dos algoritmos genéticos tradicionais, a Evolução Diferencial não utiliza operadores de cruzamento e mutação baseados em permutações. Em vez disso, novos indivíduos são gerados por operações aritméticas realizadas diretamente sobre os vetores reais que representam as soluções.

12.2 Representação da Solução

Cada indivíduo da população é representado por um vetor de números reais:

[0.42, 0.13, 0.91, 0.55, 0.27]

A ordenação crescente dos valores gera a sequência de visitação dos clientes:

Cliente 2 → Cliente 5 → Cliente 1 → Cliente 4 → Cliente 3

Essa sequência constitui uma permutação de clientes que será utilizada pelo decoder para construir as rotas.

A utilização de Random Keys permite aplicar diretamente os operadores clássicos da Evolução Diferencial sem necessidade de operadores específicos para permutações.

12.3 Decoder e Construção das Rotas

A avaliação das soluções é realizada através de um decoder responsável por converter a permutação de clientes em rotas.

O processo ocorre da seguinte forma:

1. Os clientes são percorridos na ordem definida pelo indivíduo;
2. Para cada cliente, o algoritmo tenta inseri-lo nas rotas já existentes;
3. São verificadas as restrições de capacidade e janela de tempo;
4. É escolhida a inserção que produz o menor aumento de distância;
5. Caso nenhuma inserção viável seja encontrada, uma nova rota é criada.

Para reduzir o custo computacional, apenas as rotas mais promissoras são analisadas durante a inserção. Essas rotas são selecionadas com base na proximidade entre o cliente candidato e o último cliente de cada rota.

Essa estratégia reduz significativamente o tempo de execução sem comprometer a qualidade das soluções encontradas.

12.4 Operadores Implementados

Mutação Diferencial

O operador principal da Evolução Diferencial é a mutação diferencial.

Para cada indivíduo alvo, três indivíduos distintos da população são selecionados aleatoriamente:

$$v = x_1 + F(x_2 - x_3)$$

onde:

- x_1 , x_2 e x_3 são indivíduos distintos da população;
- F é o fator de mutação diferencial;
- v é o vetor mutante gerado.

O vetor resultante é limitado ao intervalo $[0, 1]$.

Crossover Binomial

Após a mutação diferencial, é realizado o crossover binomial entre o indivíduo alvo e o vetor

mutante.

Para cada posição do vetor:

- Com probabilidade CR, o valor vem do mutante;
- Caso contrário, o valor vem do indivíduo alvo.

Além disso, é garantido que pelo menos uma posição seja herdada do mutante.

Seleção

A seleção é realizada de forma determinística.

Após gerar o vetor teste, o algoritmo compara a solução atual com a nova solução utilizando o método de Deb para tratamento de restrições.

12.5 Tratamento de Restrições – Método de Deb

O tratamento das restrições foi realizado utilizando o método de Deb.

A comparação entre duas soluções segue três regras:

1. Entre duas soluções viáveis, vence aquela com melhor função objetivo;
2. Entre uma solução viável e uma inviável, vence a solução viável;
3. Entre duas soluções inviáveis, vence aquela com menor violação total das restrições.

A violação total considera:

- excesso de capacidade;
- atraso nas janelas de tempo;
- atraso no retorno ao depósito;
- clientes repetidos;
- clientes não atendidos;
- excesso de veículos.

Essa abordagem elimina a necessidade de definir penalizações extremamente grandes e cria uma hierarquia natural onde a viabilidade é priorizada em relação à qualidade da solução.

12.6 Função Objetivo

A função objetivo utilizada segue a definição do problema:

1. Minimizar o número de veículos utilizados;
2. Minimizar a distância total percorrida.

Dessa forma, entre duas soluções viáveis:

- a solução com menor número de veículos é preferida;
- em caso de empate, vence a solução com menor distância total.

Essa estratégia está alinhada com os critérios de avaliação do VRPTW propostos por Solomon.

12.7 Parâmetros Utilizados

Os parâmetros utilizados foram:

Parâmetro	Valor Padrão
Tamanho da população (NP)	50
Número máximo de gerações	300
Fator de mutação (F)	0,60
Taxa de crossover (CR)	0,85
Limite sem melhoria	60
Semente	42
Rotas testadas por inserção	8

Os parâmetros foram escolhidos de forma empírica buscando equilíbrio entre qualidade da solução e tempo computacional.

12.8 Critério de Parada

O algoritmo encerra quando ocorre uma das seguintes condições:

- é atingido o número máximo de gerações;
- não ocorre melhoria da melhor solução durante 60 gerações consecutivas.

Esse mecanismo reduz o tempo computacional em situações onde a população já convergiu.

12.9 Análise dos Resultados

Os resultados obtidos demonstraram que a Evolução Diferencial foi capaz de gerar soluções viáveis para as instâncias avaliadas.

A representação por Random Keys permitiu adaptar naturalmente um algoritmo contínuo para um problema combinatório.

O método de Deb mostrou-se eficiente para o tratamento das restrições, direcionando a busca inicialmente para regiões viáveis e posteriormente para soluções de melhor qualidade.

A estratégia de limitar o número de rotas avaliadas durante o processo de inserção reduziu significativamente o tempo de execução, mantendo a qualidade das soluções obtidas.

Os gráficos de convergência indicaram rápida redução das violações nas primeiras gerações, seguida pela diminuição gradual do número de veículos e da distância total percorrida.

Como principal limitação, observou-se que o custo computacional do decoder ainda cresce com o tamanho das instâncias, uma vez que cada indivíduo precisa ser convertido em rotas e avaliado a cada geração.

Apesar disso, a Evolução Diferencial apresentou desempenho satisfatório para o VRPTW, produzindo soluções viáveis e competitivas dentro do tempo disponível para execução.

